





PRINCETON
UNIVERSITY





HEP analyses with gradients

How automatic differentiation can boost your analysis sensitivity

Mohamed Aly

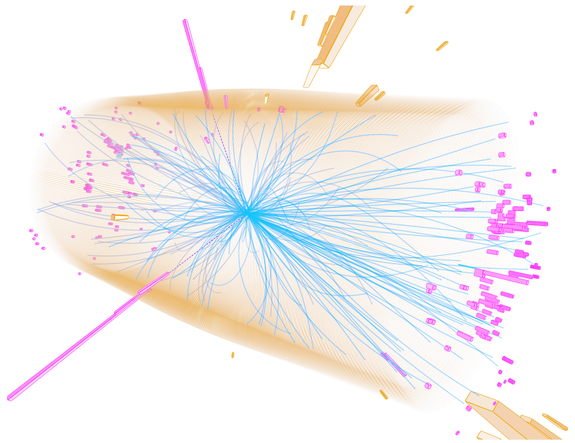
04.11.2026 @ HSF-India Mandi 2026

The case for gradients

HEP workflows

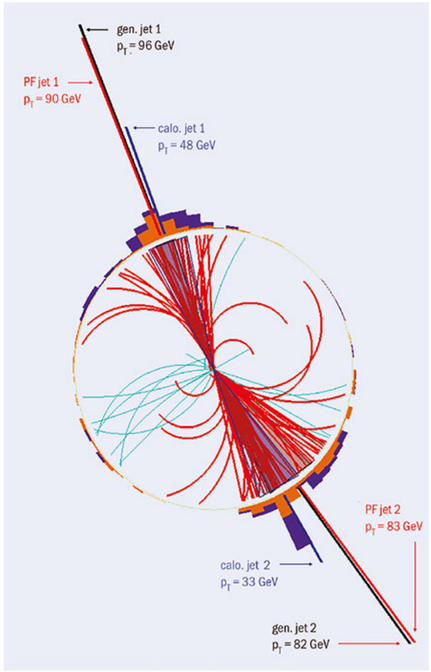
From raw data to results

Raw data

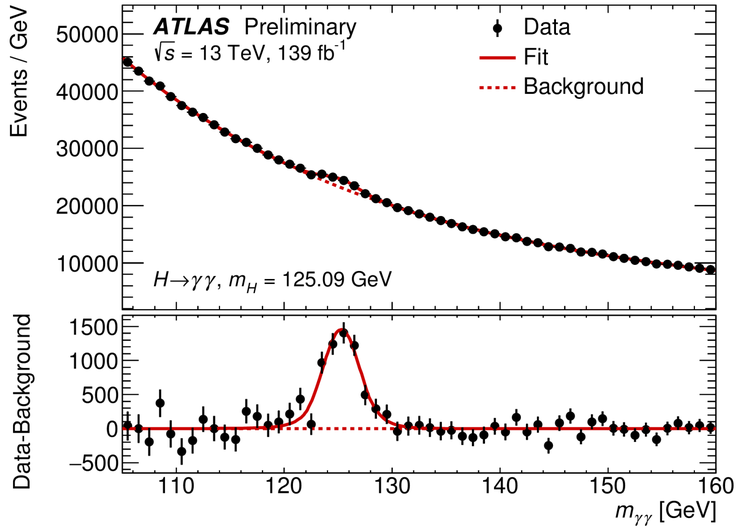


CMS event displays of Higgs to two photon candidate

Object reconstruction



Statistical analysis



HEP workflows

From raw data to results

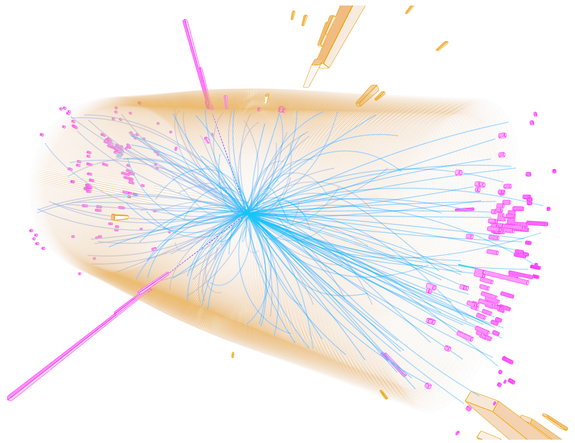
Raw data

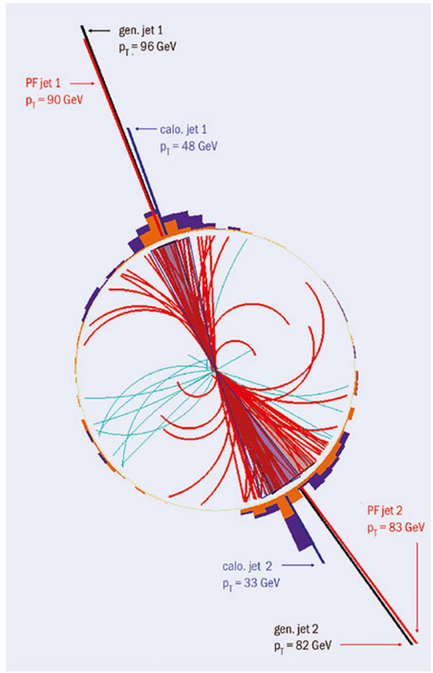
Object reconstruction

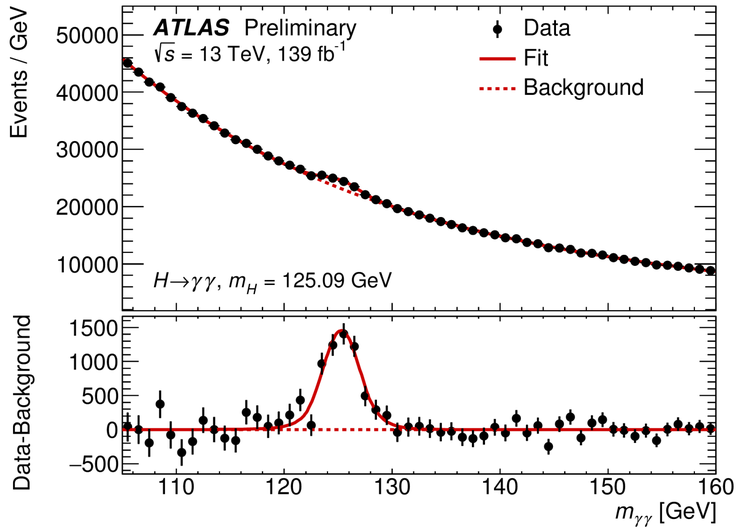
Statistical analysis

We focus on this

CMS event displays of Higgs to two photon candidate



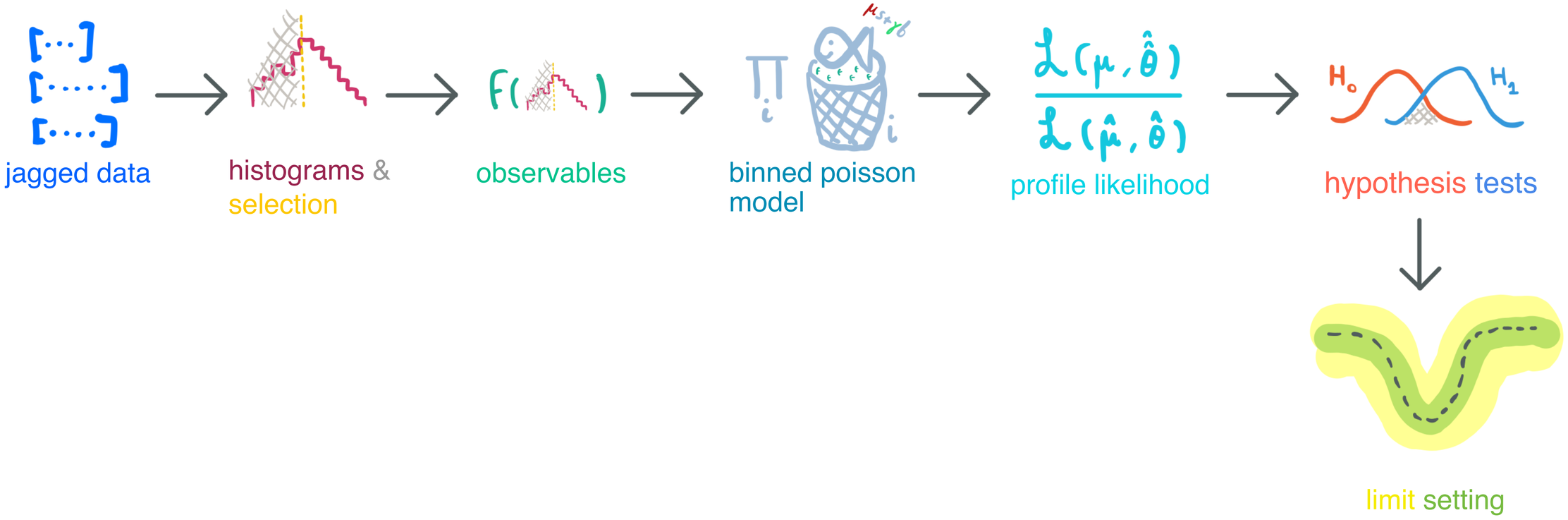




HEP workflows

The statistical analysis workflow

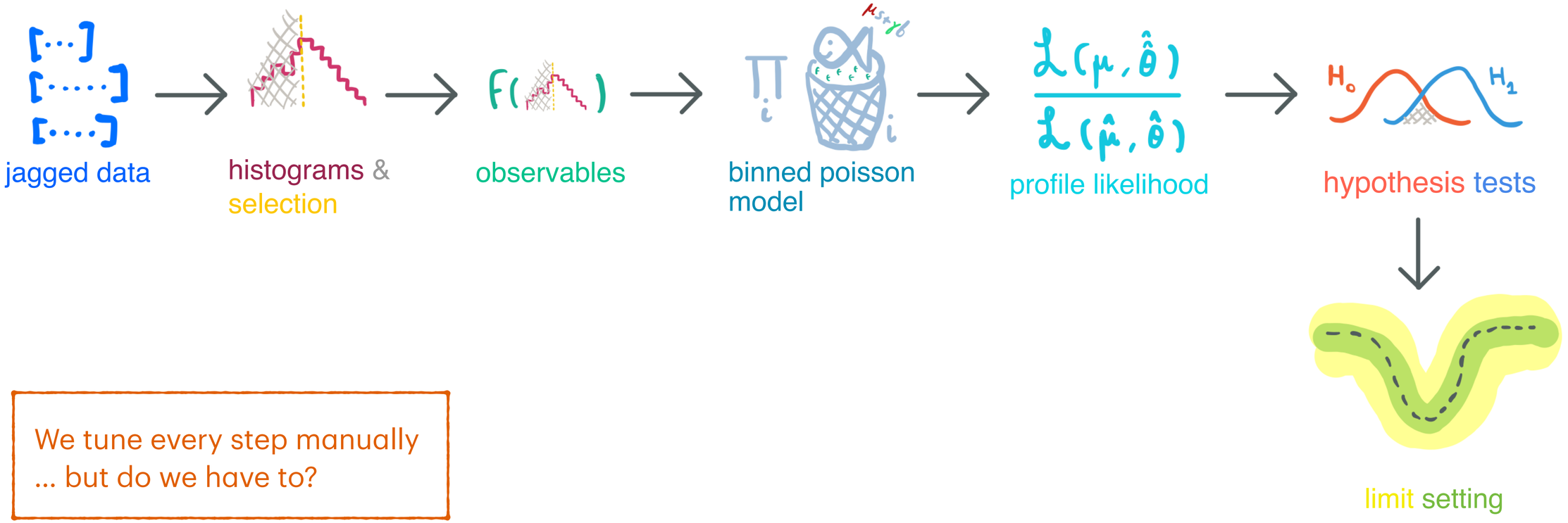
diagram from Nathan Simpson's talk



HEP workflows

The statistical analysis workflow

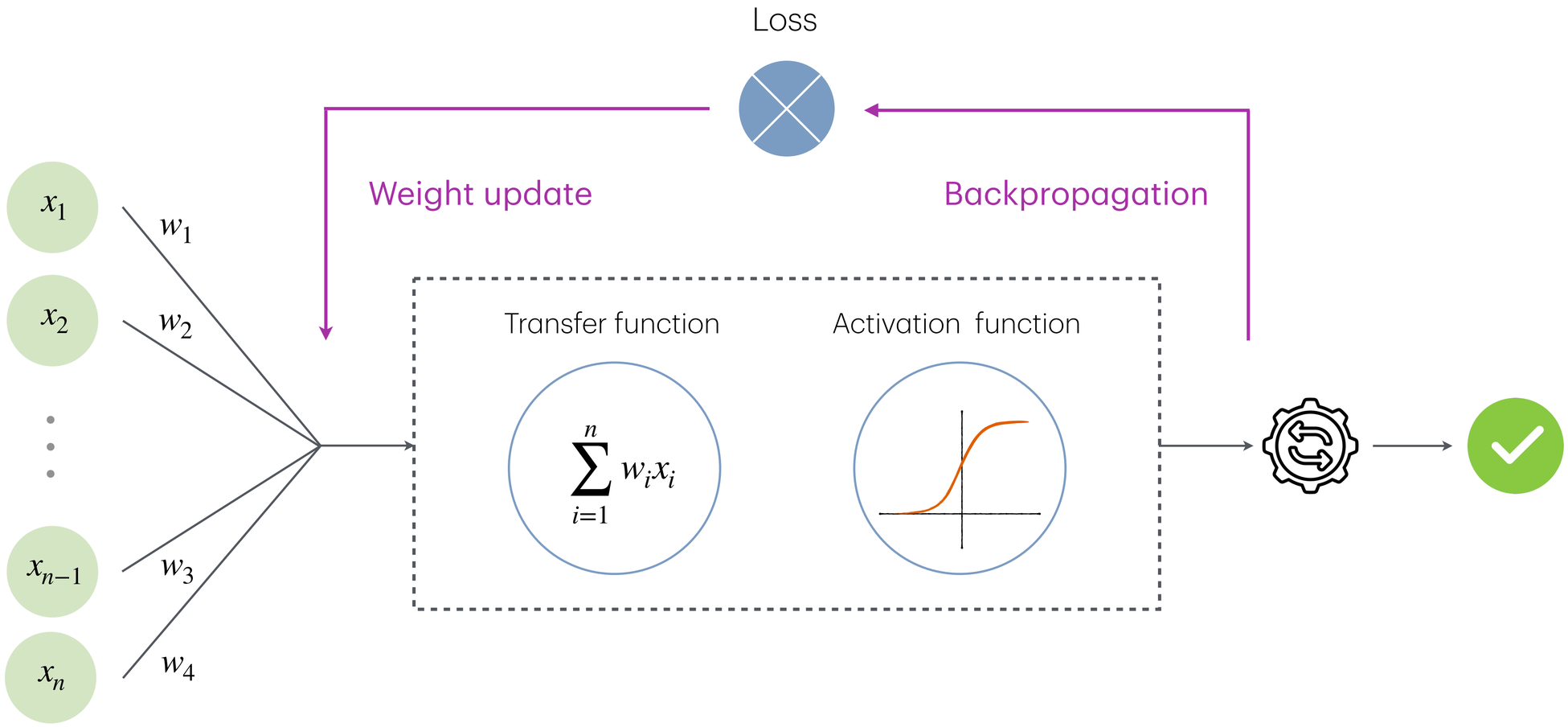
diagram from Nathan Simpson's talk



We tune every step manually
... but do we have to?

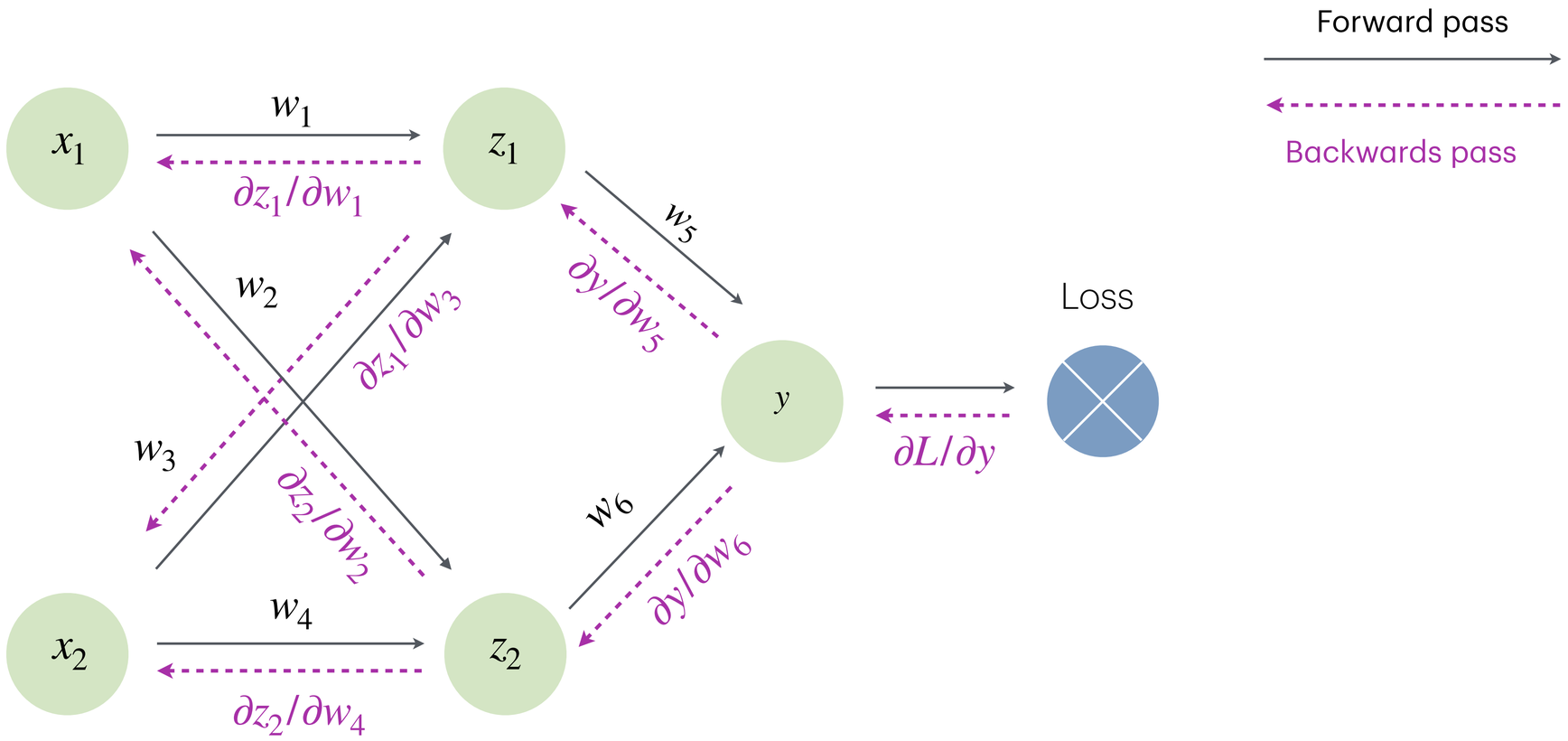
The power of gradients

In machine learning



The power of gradients

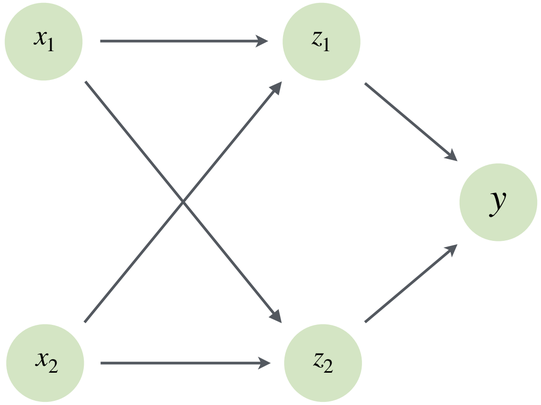
Backpropagation requires differentiation

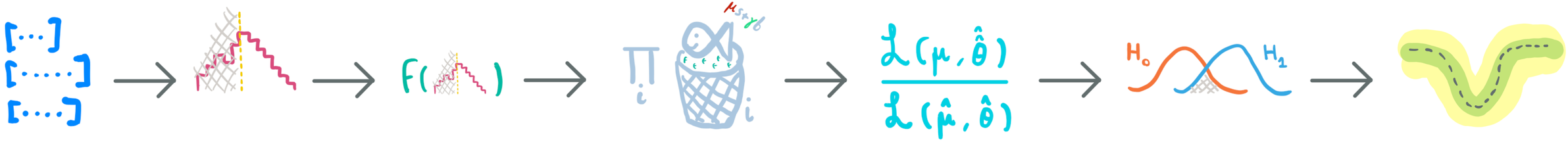


HEP workflows ft. ML

Linking up with deep learning

Bottom diagram from Nathan Simpson's talk

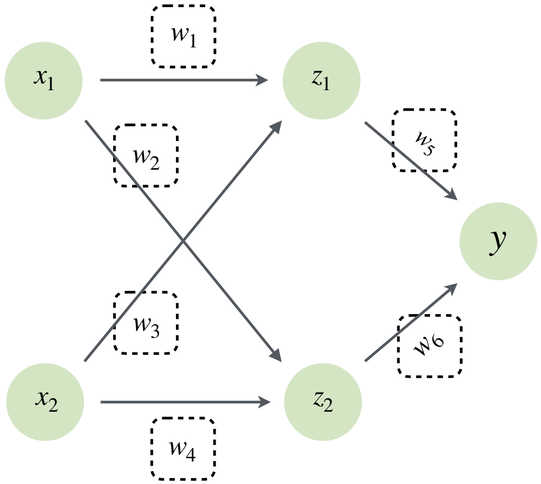




HEP workflows ft. ML

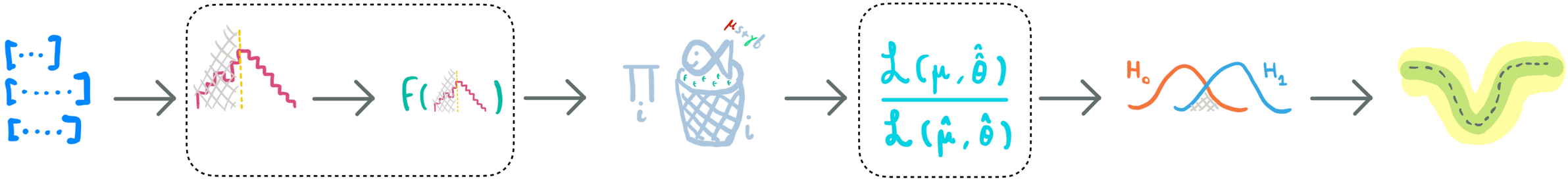
Tuneable paramaters

Bottom diagram from Nathan Simpson's talk



Cut thresholds, binning, ML, ..

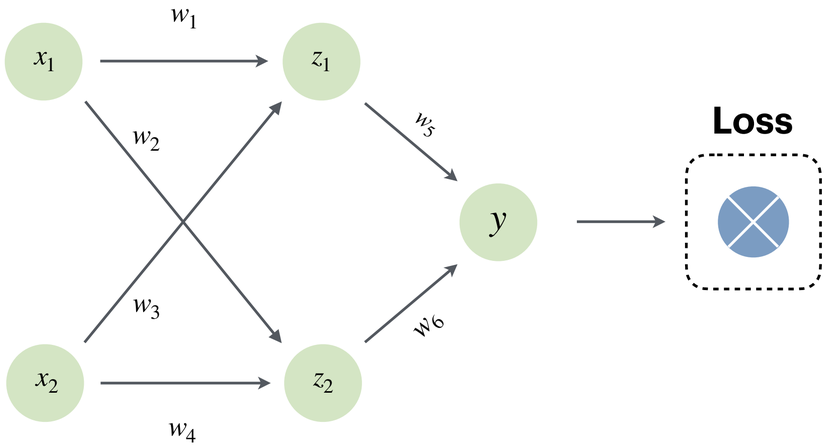
Model parameters



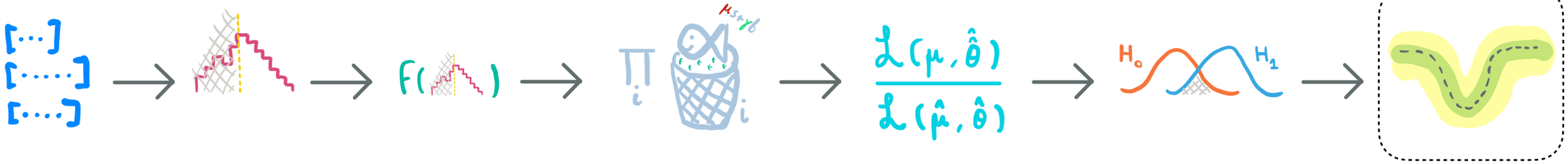
HEP workflows ft. ML

Objective to optimise w.r.t the parameters

Bottom diagram from Nathan Simpson's talk



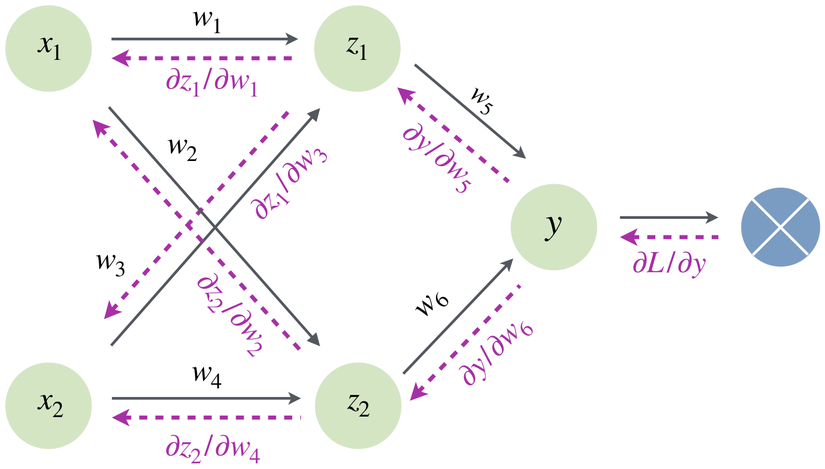
Physics objective

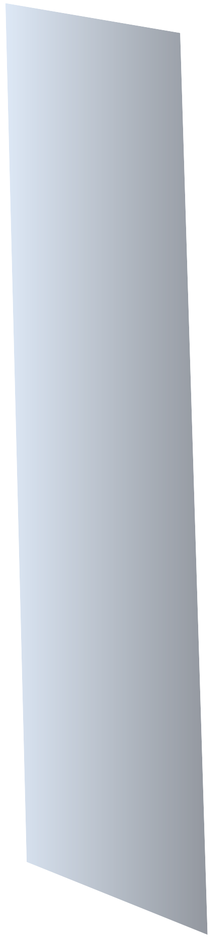


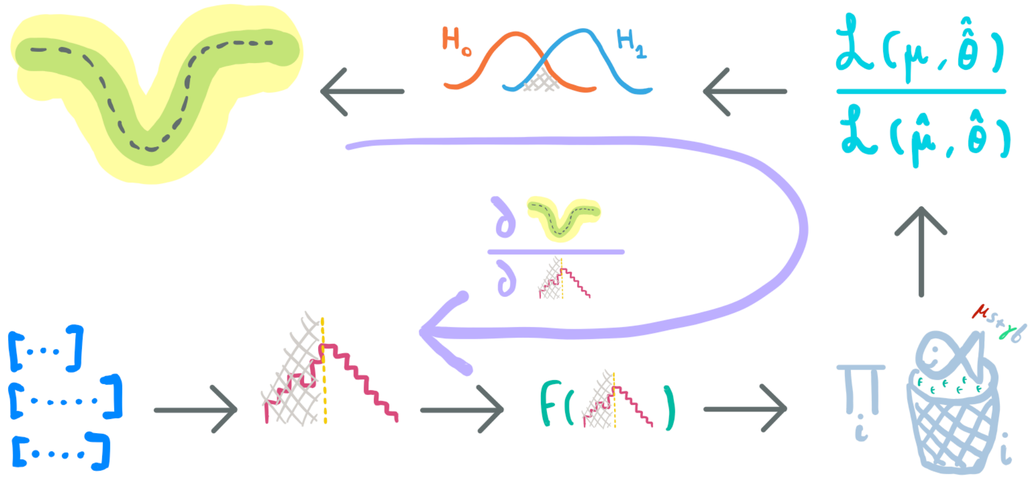
HEP workflows ft. ML

Opportunities with gradients

R.H.S. diagram from Nathan Simpson's talk





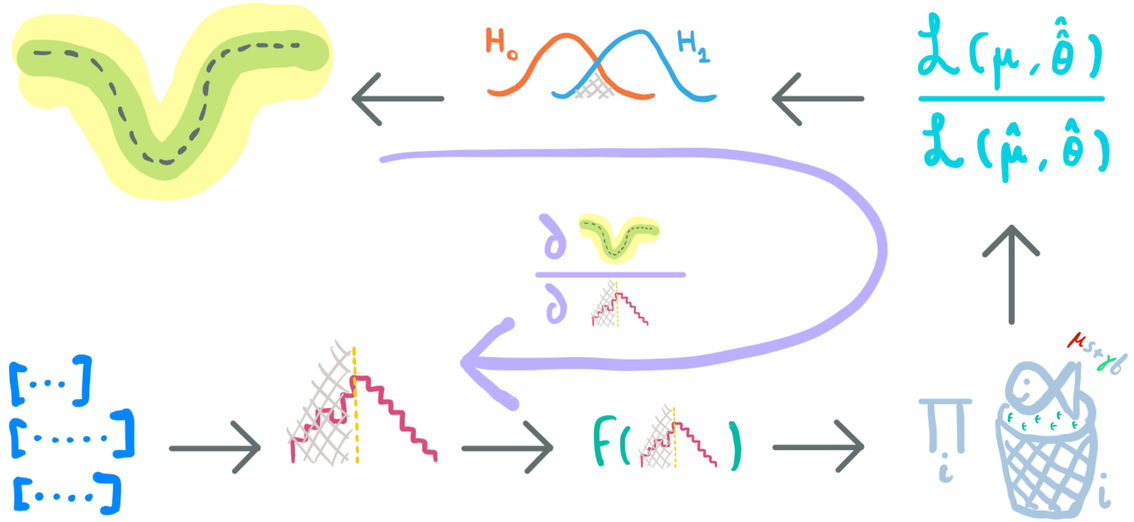


The challenge of optimisation

How do we compute the gradients

diagram from Nathan Simpson's talk

The conceptual challenge: gradients through complex code



Differentiable programming is the answer!

Differentiable Programming

What is differentiable programming?

Four ways to calculate derivates in a computer program

Manually

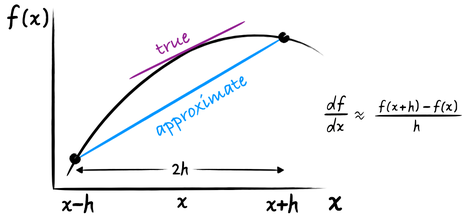


```
def f(x):
  return x*x

def df_dx(x):
  return 2*x
```

- Very difficult for complex code

Numerically

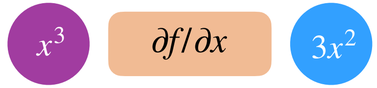



```
def f(x):
  return x*x

def df_dx(x):
  eps = 1e-6
  return (f(x+eps)-f(x))/eps
```

- Approximation errors
- Computational cost

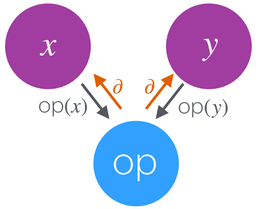
Symbolically




```
>>> from sympy import symbols, diff
>>> x = symbols('x')
>>> f = x**2
>>> diff(f, x)
2*x
```

- Expression swell
- Closed-form only

Automatically




```
def f_df(x):
    (v, dv) = (x, 1)
    (z, dz)  = (v*v, v*dv + v*dv)
    return (z, dz)
```

- No analytical expressions
- Exact gradients with small computational cost

What is differentiable programming?

Four ways to calculate derivates in a computer program

Manually

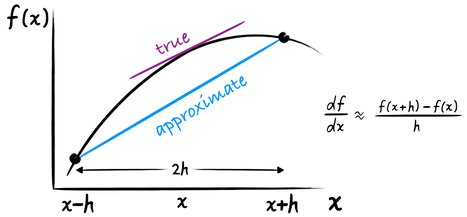


```
def f(x):
  return x*x

def df_dx(x):
  return 2*x
```

- Very difficult for complex code

Numerically

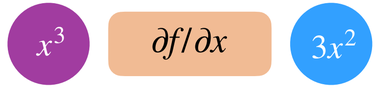



```
def f(x):
  return x*x

def df_dx(x):
  eps = 1e-6
  return (f(x+eps)-f(x))/eps
```

- Approximation errors
- Computational cost

Symbolically

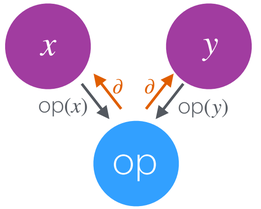



```
>>> from sympy import symbols, diff
>>> x = symbols('x')
>>> f = x**2
>>> diff(f, x)
2*x
```

- Expression swell
- Closed-form only

Automatically

Backprop




```
def f_df(x):
    (v, dv) = (x, 1)
    (z, dz)  = (v*v, v*dv + v*dv)
    return (z, dz)
```

- No analytical expressions
- Exact gradients with small computational cost



An accelerated, more efficient NumPy with gradients

Automatic Differentiation

```
>>> import jax
>>> f = lambda x: x**3 + 2 * x**2 - 3 * x + 1
>>> jax.grad(f)(1.0)    # evaluate at x=1
4.0
>>> jax.grad(jax.grad(f))(1.0)
10.0
>>> jax.grad(jax.grad(jax.grad(f)))(1.0)
6.0
>>> jax.grad(jax.grad(jax.grad(jax.grad(f))))(1.0)
0.0
```

Differentiate w.r.t any PyTree

Efficient JIT compilation

```
>>> import jax, jax.numpy as jnp
>>> def selu(x):
...     return 1.05 * jnp.where(x > 0, x, 1.67 * jnp.exp(x) - 1.67)
>>> x = jnp.arange(5., dtype=jnp.float32)
>>> jax.jit(selu)(x)
[0.     1.1935 2.1 3.15 4.2]
```

JIT compile functions into XLA-executables

Automatic (explicit) Vectorisation

```
>>> import jax, jax.numpy as jnp
>>> def distance(x, y): return jnp.sqrt(jnp.sum((x - y)**2))
>>> x = jnp.array([[1., 2.], [3., 4.], [5., 6.]])
>>> y = jnp.array([0., 0.])
>>> jax.vmap(distance, in_axes=(0, None))(x, y)
[2.236068 5. 7.81025]
```

Auto-batch scalar code (not just array ops)

Multi-device parallelisation

```
>>> import jax, jax.numpy as jnp

>>> def f(x): return x * 2
>>> n = jax.device_count()
>>> xs = jnp.arange(n * 3).reshape(n, 3)   # if n=4 → shape (4, 3)
>>> jax.pmap(f)(xs)                        # runs f on each GPU in parallel
[[ 0  2  4]
 [ 6  8 10]
 [12 14 16]
 [18 20 22]]
```

Split data across devices and parallelise

The path to HEP gradients

The path to gradients

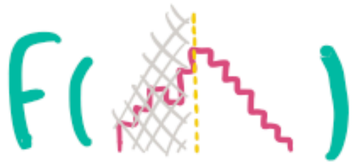
The challenges

Non-differentiable operations and internal optimisations

1. Discrete operations



2. Histograms



3. Optimising the likelihood

$$\frac{\mathcal{L}(\mu, \hat{\hat{\theta}})}{\mathcal{L}(\hat{\mu}, \hat{\theta})}$$

Differentiable Relaxations

Making discrete primitives differentiable

Non-differentiable operations and internal optimisations

1. Discrete operations



e.g. applying a threshold, sorting

2. Histograms



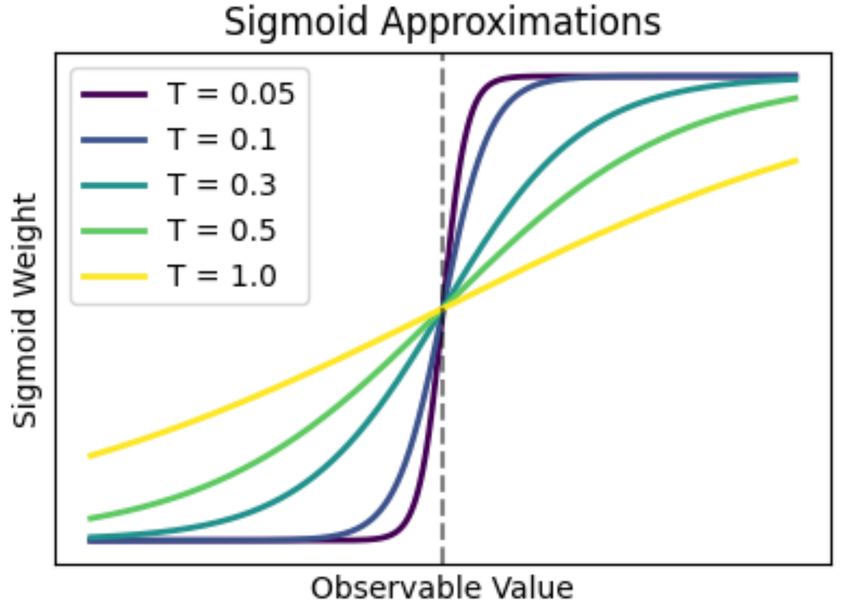
3. Optimising the likelihood

$$\frac{\mathcal{L}(\mu, \hat{\theta})}{\mathcal{L}(\hat{\mu}, \hat{\theta})}$$

Differentiable Relaxations

Making discrete primitives differentiable

We can approximate (smoothen) the operations with continuous functions → continuous relaxations

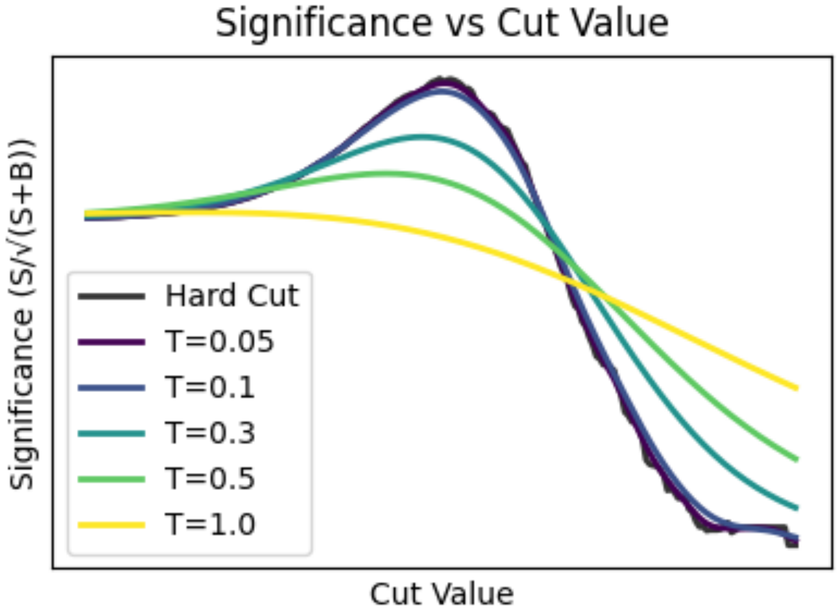


Smoothing controlled by the "Temperature" (T)

Differentiable Relaxations

Making discrete primitives differentiable

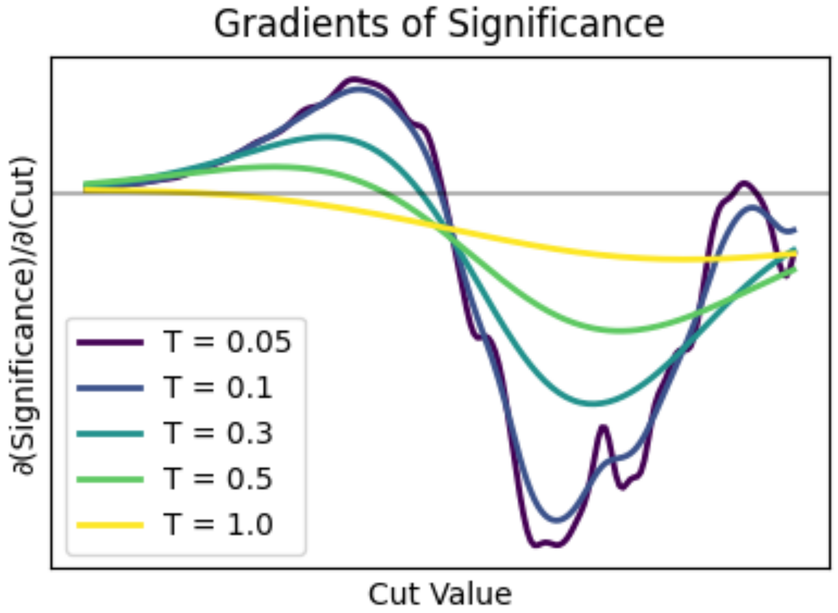
Optimal significance value is less robust against softening than its location (i.e. cut value)



Differentiable Relaxations

Making discrete primitives differentiable

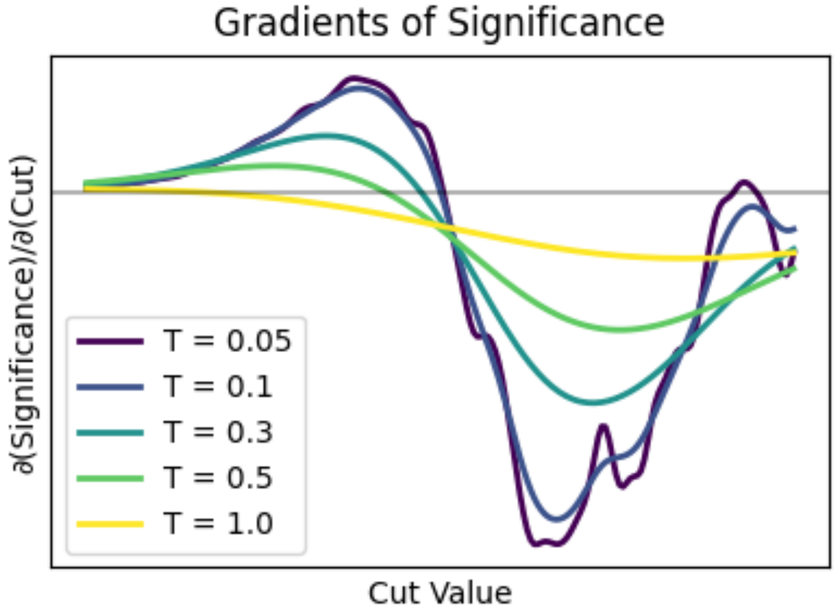
A trade-off: faithfulness to a hard threshold and stability of in gradient evaluation



Differentiable Relaxations

Making discrete primitives differentiable

Compromise: we keep all events → dataset size stays large



The path to gradients

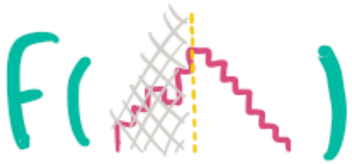
The challenges

Non-differentiable operations and internal optimisations

1. Discrete operations



2. Histograms



3. Optimising the likelihood

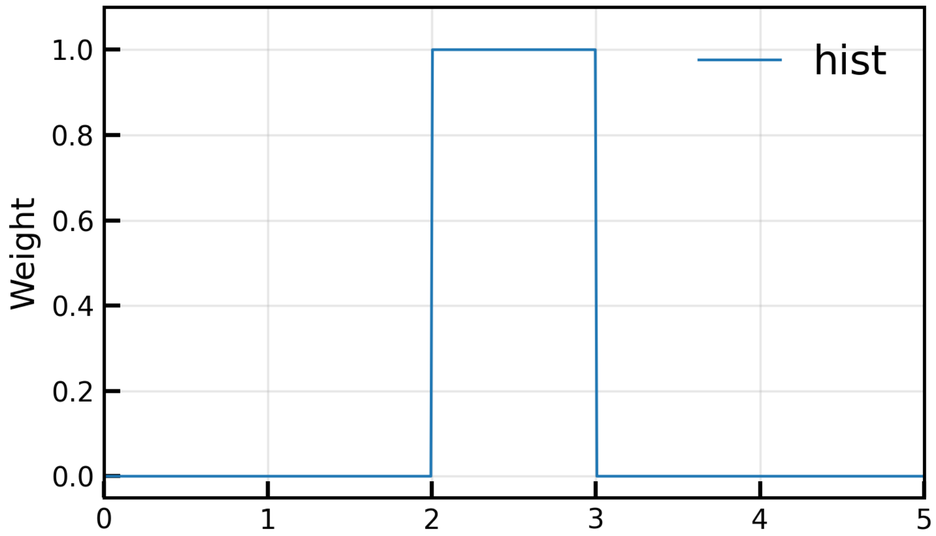
$$\frac{\mathcal{L}(\mu, \hat{\theta})}{\mathcal{L}(\hat{\mu}, \hat{\theta})}$$

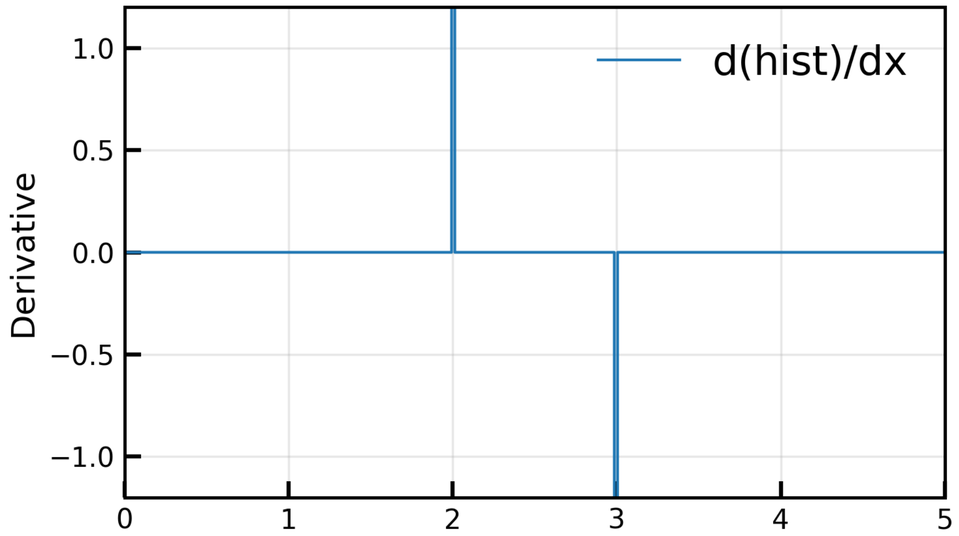
Differentiable Relaxations

Making histograms differentiable

Histograms are non-differentiable data structures

Vary data point: land in same bin, 0 gradient, land in another bin, infinite gradient





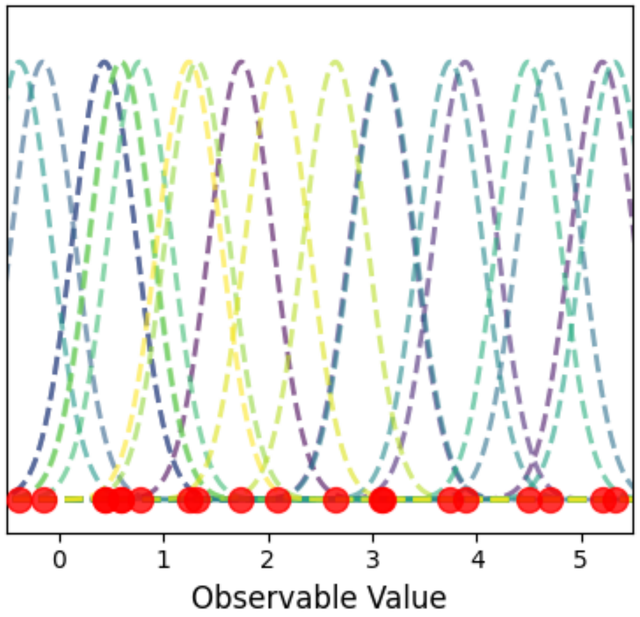
Differentiable Relaxations

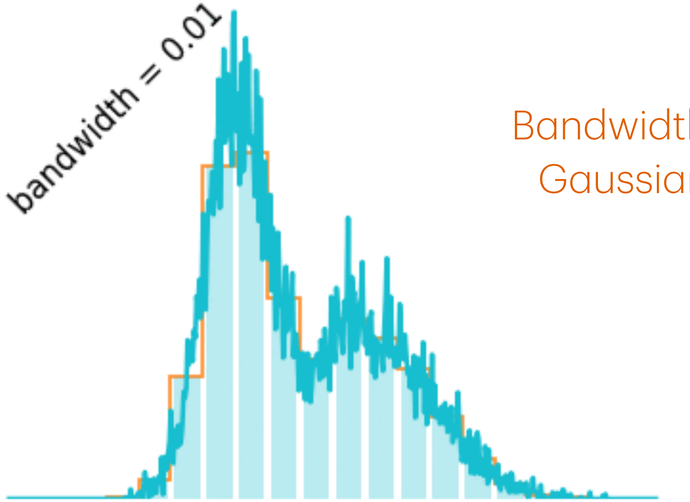
Making histograms differentiable

diagrams from by Nathan Simpson's tutorial

Relaxation: e.g. binned kernel density estimate (bKDE)

Step 1: Define 1 Gaussian per-data point, and sum all Gaussians





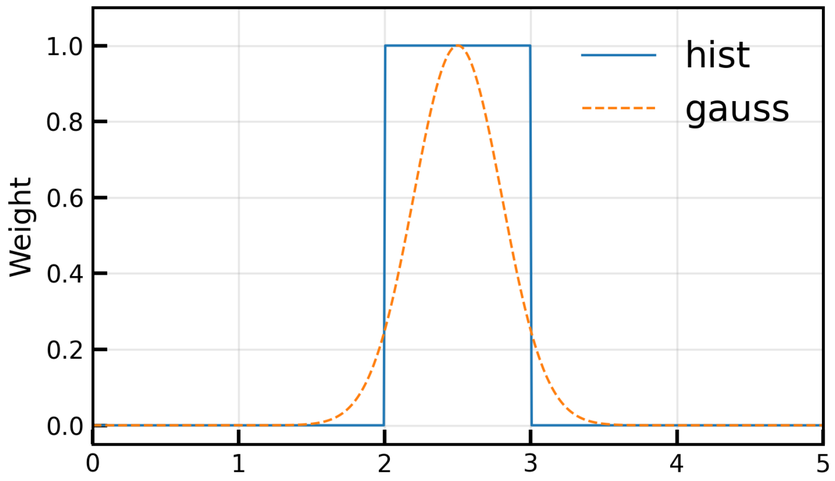
Bandwidth controls Gaussian widths

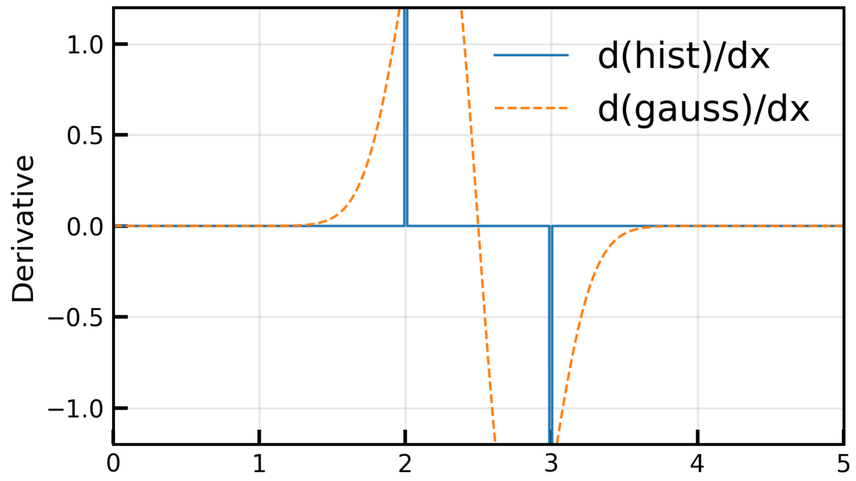
Differentiable Relaxations

Making histograms differentiable

Relaxation: binned kernel density estimate (bKDE)

Step 2: Integrate the Gaussian between bin-edges to get bin content



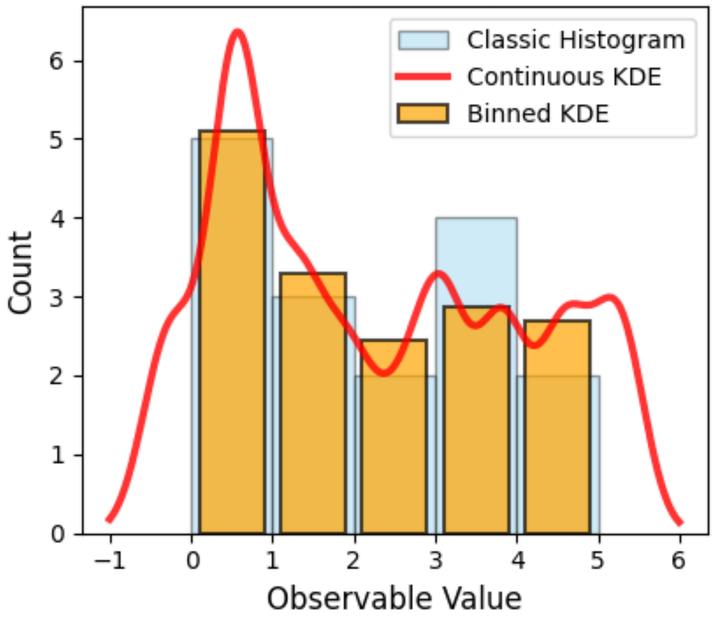


Differentiable Relaxations

Making histograms differentiable

Relaxation: binned kernel density estimate (bKDE)

Step 3: Differentiate each bin with respect to each data-point



Great tutorial on this by Nathan Simpson

The path to gradients

The challenges

Non-differentiable operations and internal optimisations

1. Discrete operations



2. Histograms



3. Optimising the likelihood

$$\frac{\mathcal{L}(\mu, \hat{\hat{\theta}})}{\mathcal{L}(\hat{\mu}, \hat{\theta})}$$

Fixed-Point Differentiation

Optimising the analysis while fitting to data

Caveat: the significance requires performing fits

Significance is a function of:

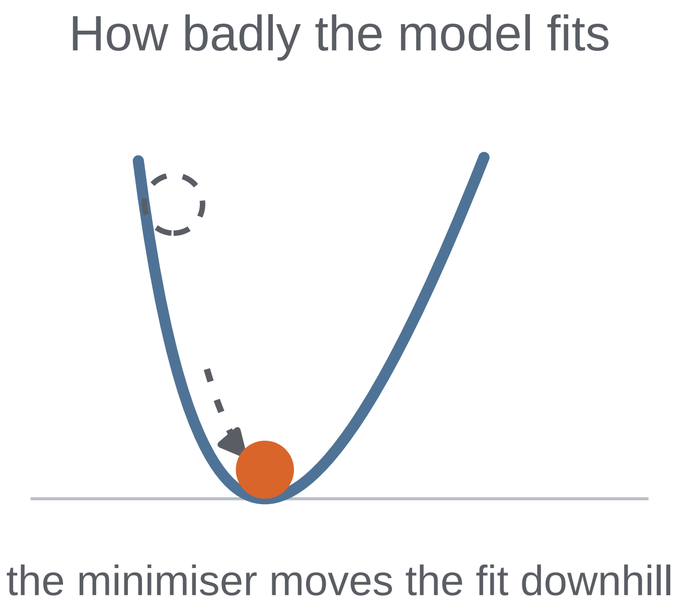
$$\frac{\mathcal{L}(\mu, \hat{\hat{\theta}})}{\mathcal{L}(\hat{\mu}, \hat{\theta})}$$

Run a nested minimiser for each iteration in parameter optimisation?

Implicit Function Theorem

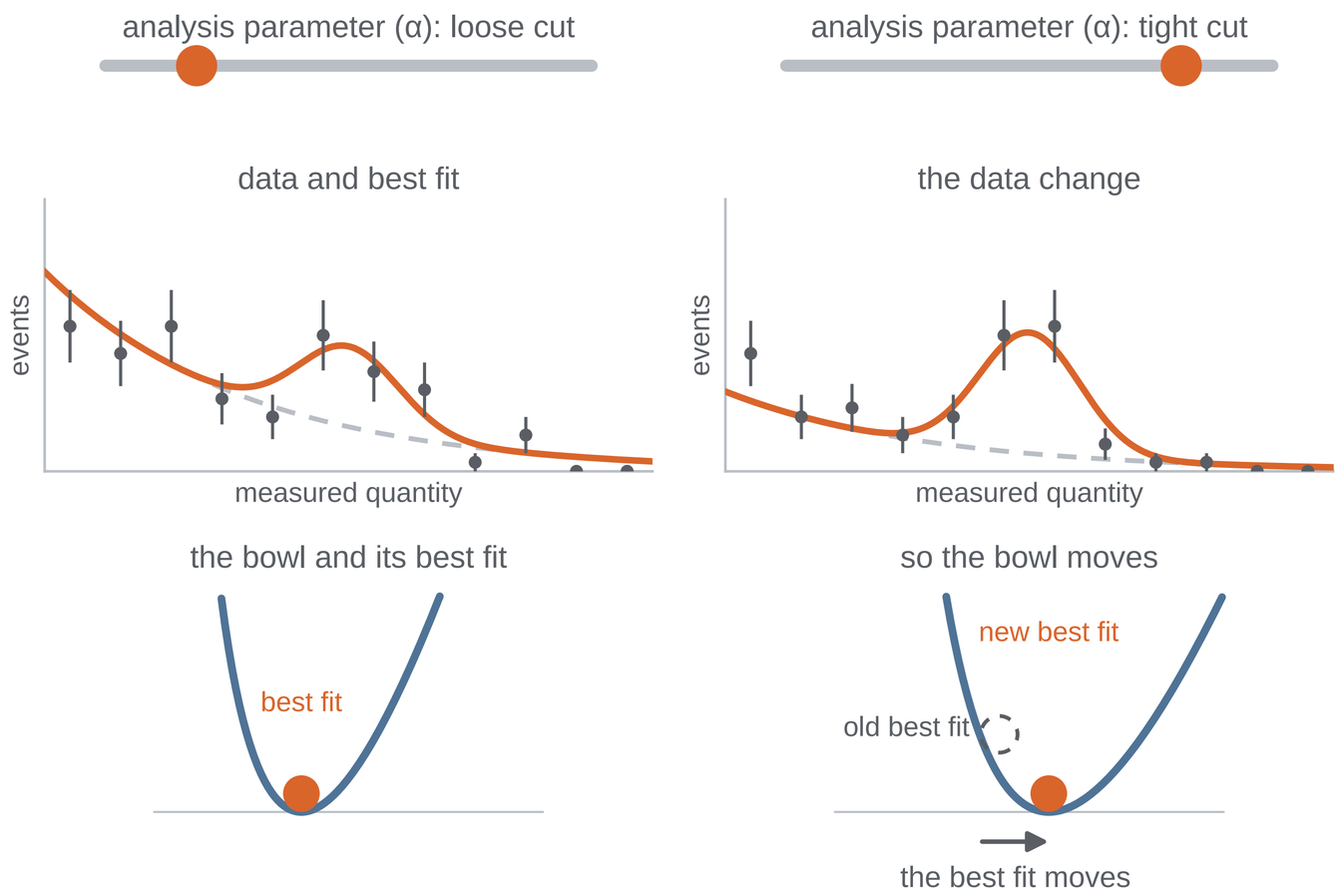
Visual guide: the fit as a ball in a bowl





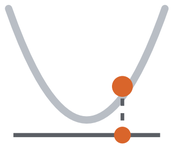
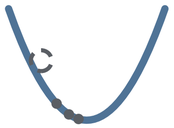
Implicit Function Theorem

Visual guide: a cut can change fit result



Implicit Function Theorem

Visual guide: ball in a bowl in relation to fitting

In the picture		In the fit
 the bowl	⟷	how badly the model fits the data (the negative log-likelihood)
 the ball's position	⟷	the fit parameters θ
 rolling downhill	⟷	the minimiser's iterations
 resting at the bottom	⟷	the best fit: the gradient of the likelihood is zero

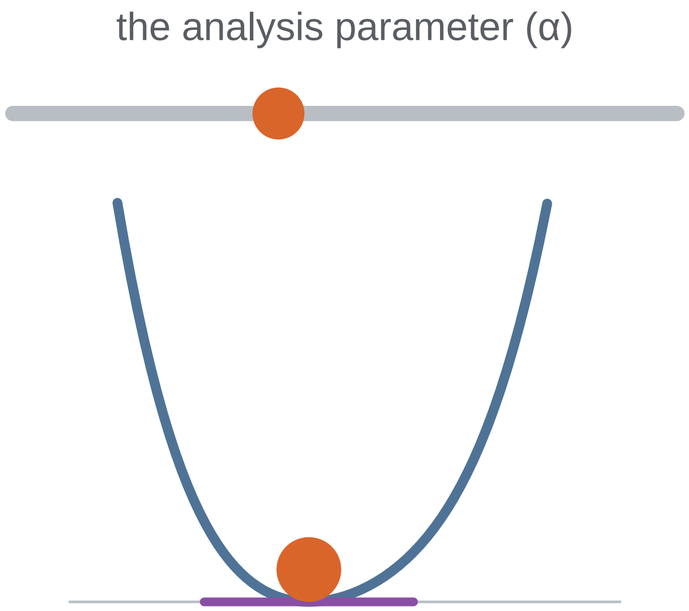
Implicit Function Theorem

Visual guide: ball in a bowl in relation to fitting

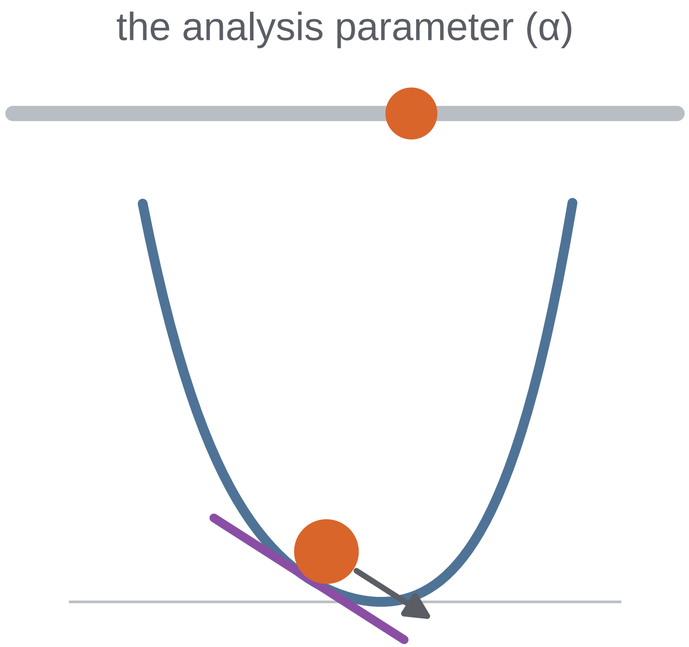
In the picture		In the fit
 the analysis parameter	⟷	α, a parameter we do not fit: an analysis choice, such as a cut
 changing the analysis parameter changes the loss	⟷	α changes the data, so the likelihood changes
 the rate of change of the loss's slope with the analysis parameter	⟷	$\partial F/\partial \alpha$: the rate of change with α of the loss's slope at the old best fit
 the rate of change of the loss's slope with the fit parameter	⟷	$\partial F/\partial \theta$: the rate of change of the loss's slope with θ (the likelihood's curvature)

Implicit Function Theorem

Visual guide: slope of the loss w.r.t analysis parameters changes



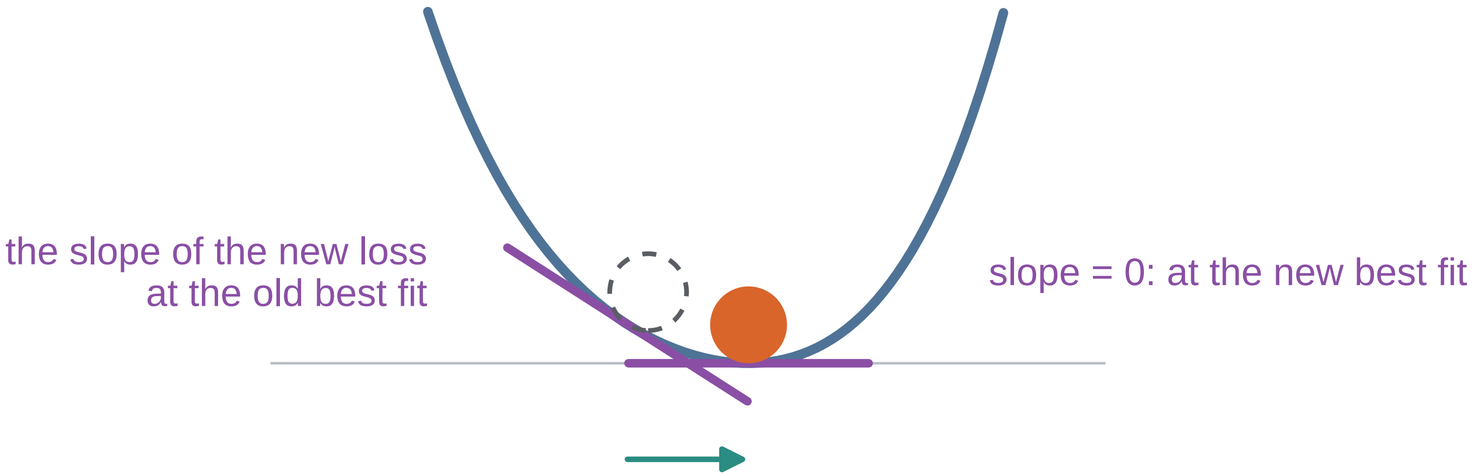
slope = 0: at the best fit



slope ≠ 0: the best fit must move

Implicit Function Theorem

Visual guide: rate of change of slop determines shift

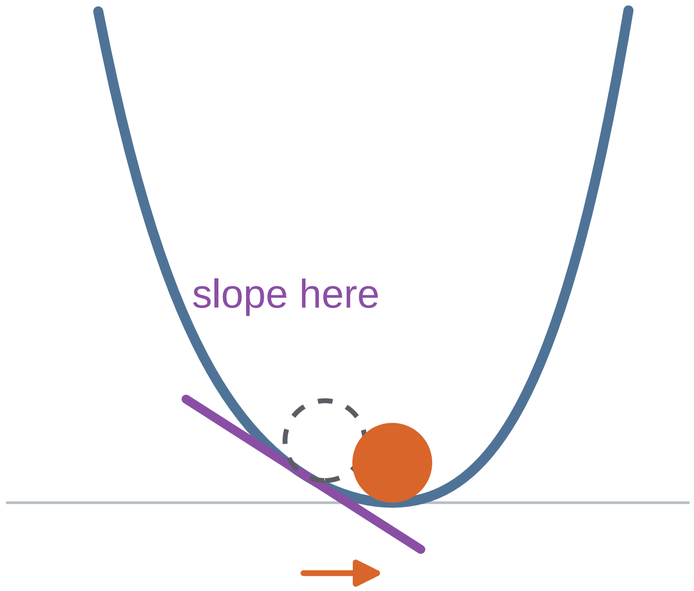


as the fit parameter moves along the curve, the slope shrinks

the rate of change of the loss's slope with the fit parameter: we divide by this

Implicit Function Theorem

Visual guide: rate of change of slop determines shift



shift = slope ÷ that rate of change

At the old best fit:

1. the slope of the new loss here
2. the rate of change of the loss's slope with the fit parameter
3. shift = slope ÷ that rate of change

No refit needed.

Implicit Function Theorem

Visual guide: in maths

Both are measured at the old best fit.

$$\frac{d\theta^*}{d\alpha} = -\frac{\partial F}{\partial \alpha}\left(\frac{\partial F}{\partial \theta^*}\right)^{-1}$$

the rate of change of the best fit with the analysis parameter

the rate of change of the loss's slope here with the analysis parameter

the rate of change of the loss's slope with the fit parameter (we divide by it)

Implicit Function Theorem

The derivation

Solution from Implicit function theorem (IFT) → fixed-point differentiation

non-fit parameters α

fit parameters θ

Likelihood function $\mathscr{L} = \mathscr{L}(D(\alpha), \theta)$ (Implicit: $D(\alpha)$; Explicit: θ)

Likelihood condition

$$F(\alpha, \theta)\,|_{\theta_0} = \partial_\theta \mathscr{L}(D(\alpha), \theta)\,|_{\theta=\theta_0} = 0$$

Implicit function theorem

If we have a function with a ROOT

$$F(\alpha_0, \theta_0) = 0$$

There exists an θ^* such that

$$\theta_0 = \theta^*(\alpha_0) \qquad F(\alpha, \theta^*(\alpha)) = 0$$

Therefore

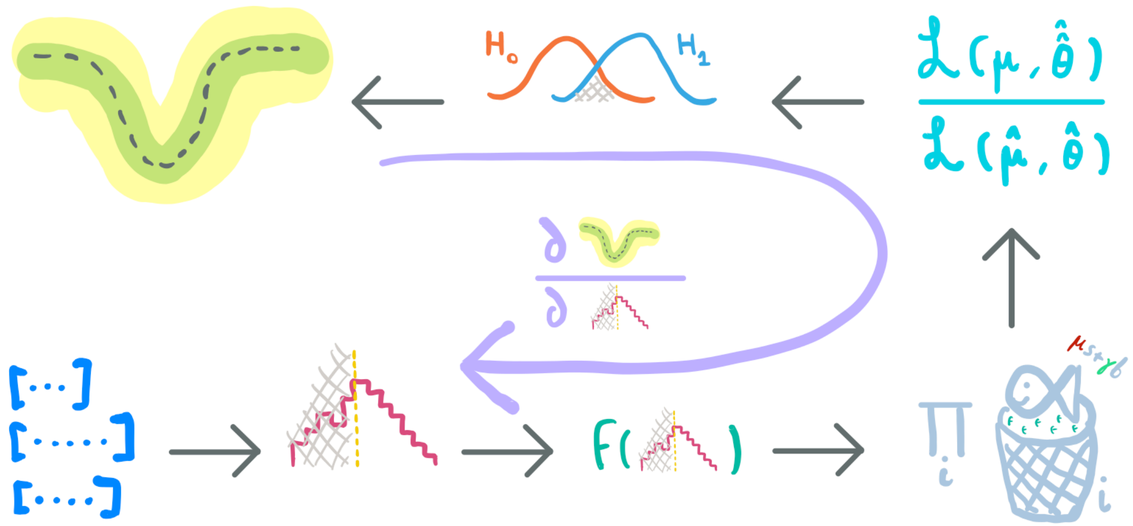
$$\frac{dF}{d\alpha} = 0 \xrightarrow{\text{for } \alpha \sim \alpha_0} \frac{d\theta^*}{d\alpha} = -\frac{\partial F}{\partial \alpha}\left(\frac{\partial F}{\partial \theta^*}\right)^{-1}$$

What do we do with all this?

Optimising the analysis while fitting to data

diagram from Nathan Simpson's talk

With the gradient of the significance with respect to all parameters
→ we can optimise all parameters to maximise significance



Case study: a $Z' \to t\bar{t}$ analysis

Differentiating through CMS open data

Diagram from CMS open-data workshop

- Searching for a hypothetical (heavy) BSM particle called Z' [arXiv:1810.05905]
  - Produced in proton-proton collisions, decays into a pair of top quarks

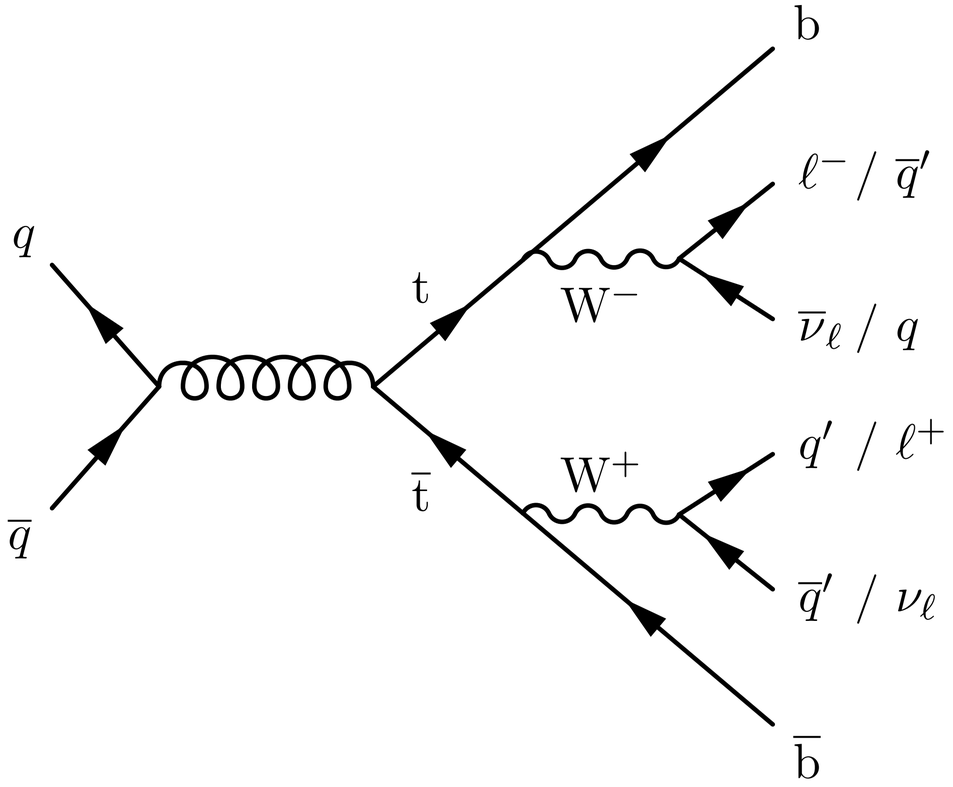


(resolved) (boosted)

The simplified analysis

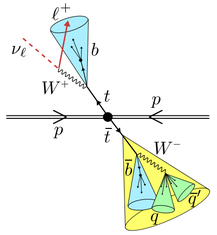
Based on CMS open-data workshop [2024]

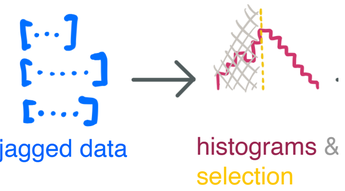
Main background: the production of top-quark pairs from the SM



The simplified analysis

Based on CMS open-data workshop [2024]

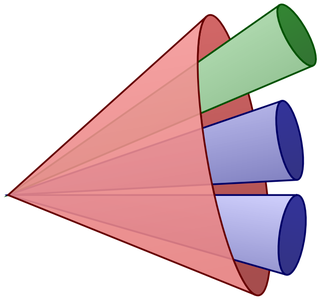




Selecting the signal and dumping the background







Exactly 1 muon

MET > 50 GeV

> 1 b-tagged jet

Proxy to b-quark

Exactly 1 fat-jet

Proxy to top-quark

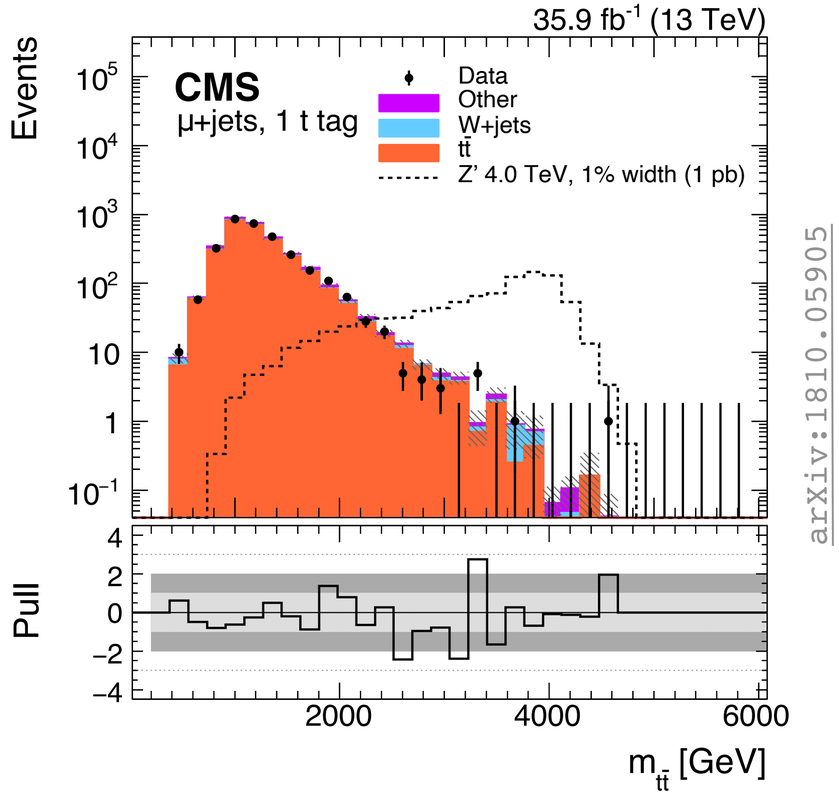
H_T = MET + muon momentum > 150 GeV

The simplified analysis

Based on CMS open-data workshop [2024]

observables

Building a discriminating observable: mass of the $t\bar{t}$ system or others



The simplified analysis

Based on CMS open-data workshop [2024]



profile likelihood

$$\frac{\mathcal{L}(\mu, \hat{\theta})}{\mathcal{L}(\hat{\mu}, \hat{\theta})}$$



hypothesis tests

Perform a binned profile likelihood fit to extract signal significance [p-value]

$$p(\vec{n}, \vec{a} \mid \vec{k}, \vec{\theta}) = \prod_{i \in N_{ch}} \text{Pois}(n_i \mid \nu_i(\vec{k}, \vec{\theta})) \cdot \prod_{j \in N_\theta} c_j(a_j \mid \theta_j)$$

Bin-by-bin Poisson

Priors of fit parameters

Optimisation

Physics objectives and parameters to optimise

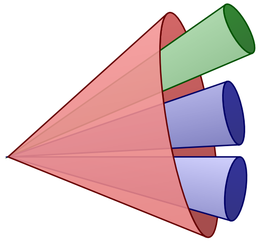
Compute the derivative with respect to free parameters



Exactly 1 muon

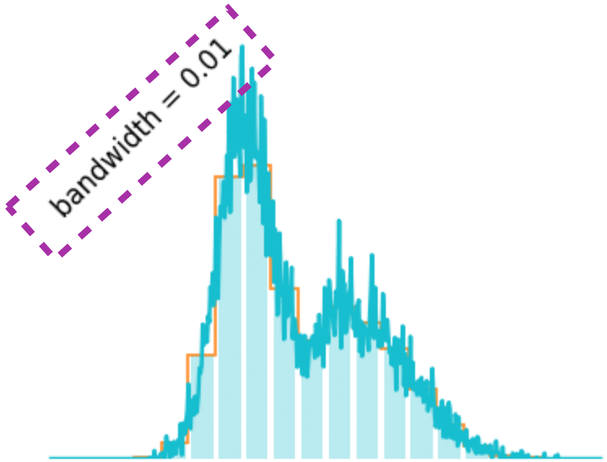


MET > 50 GeV



> 1 b-tagged jet

H_T = MET + muon momentum > 150 GeV



Optimisation

Physics objectives and parameters to optimise

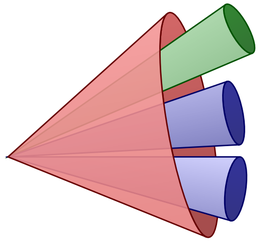
Compute the derivative with respect to free parameters



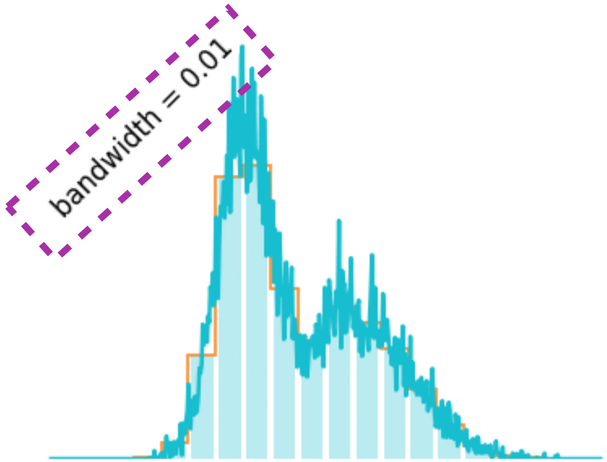
Exactly 1 muon



MET > 50 GeV



> 1 b-tagged jet



H_T = MET + muon momentum > 150 GeV

Use gradients to optimise p-value with respect to free parameters

.backup

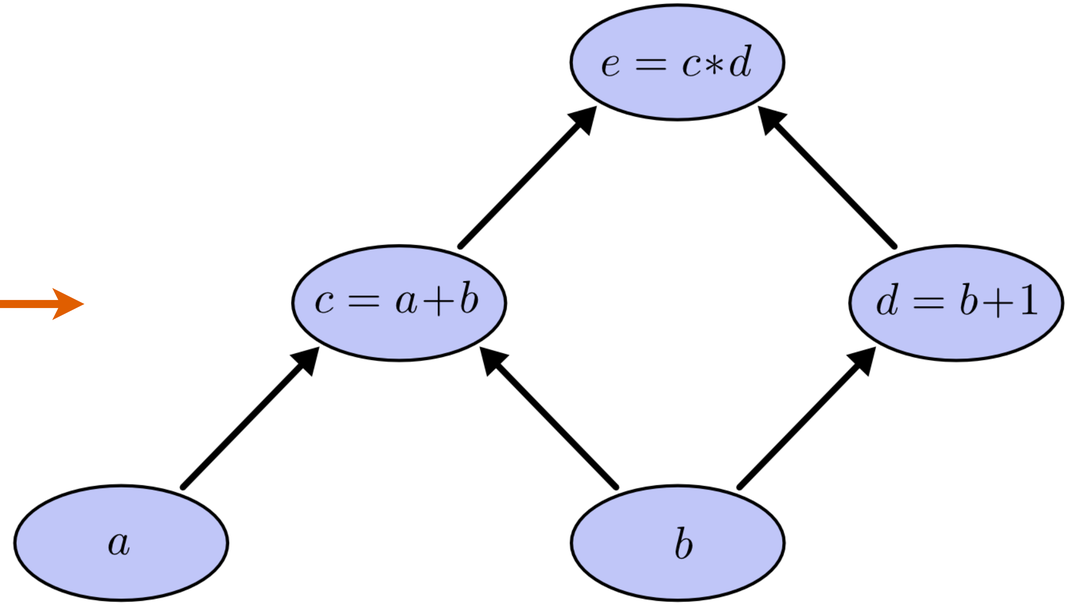
Automatic differentiation

Computational graphs

Example from this blog

Mathematical expression → elementary operations with known derivatives

$$e = (a + b) * (b + 1)$$



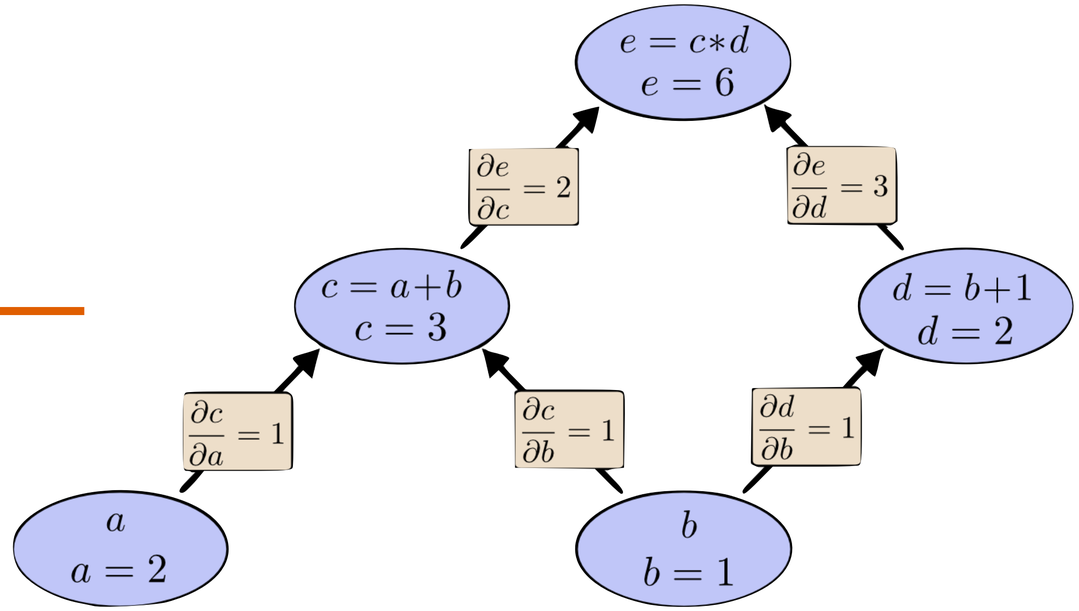
Automatic differentiation

Derivatives on the graph

Example from this blog

Compute partial derivatives along each edge

$$\left(\frac{\partial e}{\partial a}, \frac{\partial e}{\partial b}\right) = (?, ?)$$



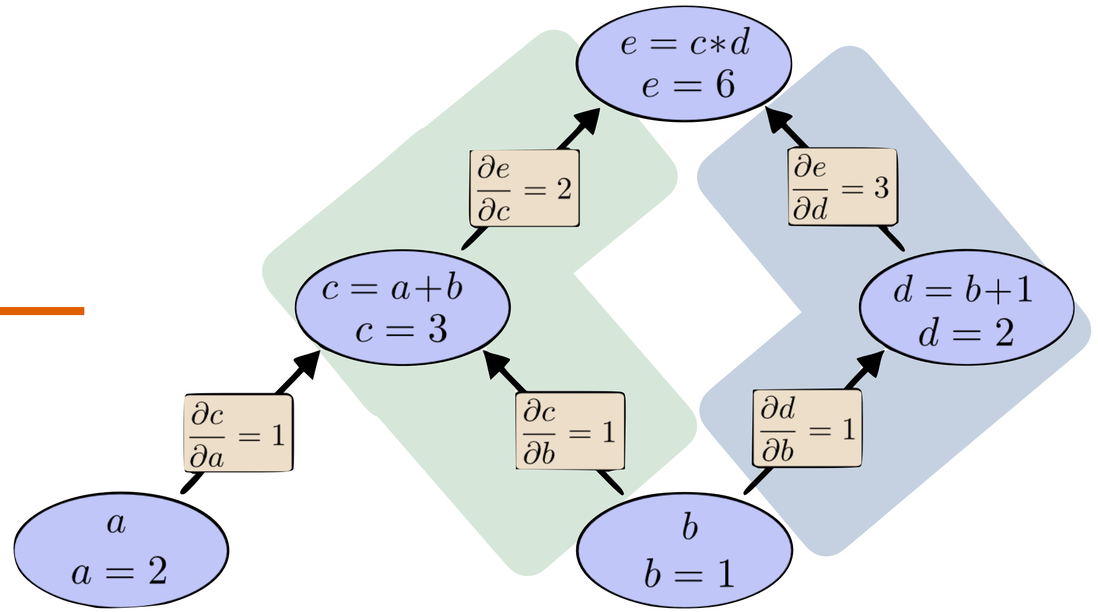
Automatic differentiation

Derivatives on the graph

Example from this blog

Multiply derivatives in a path, and sum over paths

$$\frac{\partial e}{\partial b} = \frac{\partial e}{\partial c} * \frac{\partial c}{\partial b} + \frac{\partial e}{\partial d} * \frac{\partial d}{\partial b}$$



Automatic differentiation

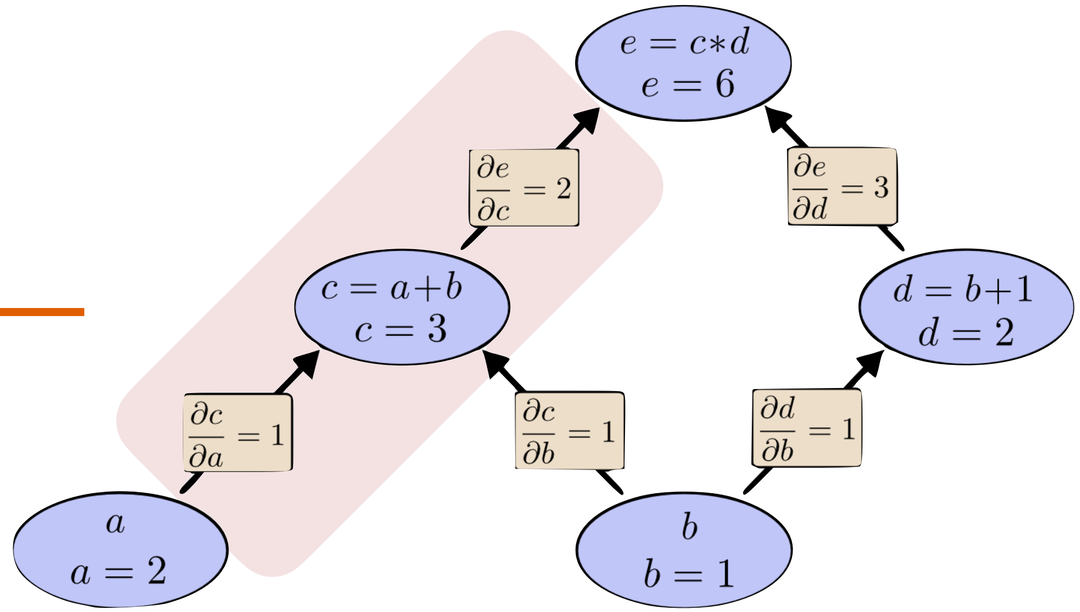
Derivatives on the graph

Example from this blog

Multiply derivatives in a path, and sum over paths

$$\frac{\partial e}{\partial b} = \frac{\partial e}{\partial c} * \frac{\partial c}{\partial b} + \frac{\partial e}{\partial d} * \frac{\partial d}{\partial b}$$

$$\frac{\partial e}{\partial a} = \frac{\partial e}{\partial c} * \frac{\partial c}{\partial a}$$



Automatic differentiation

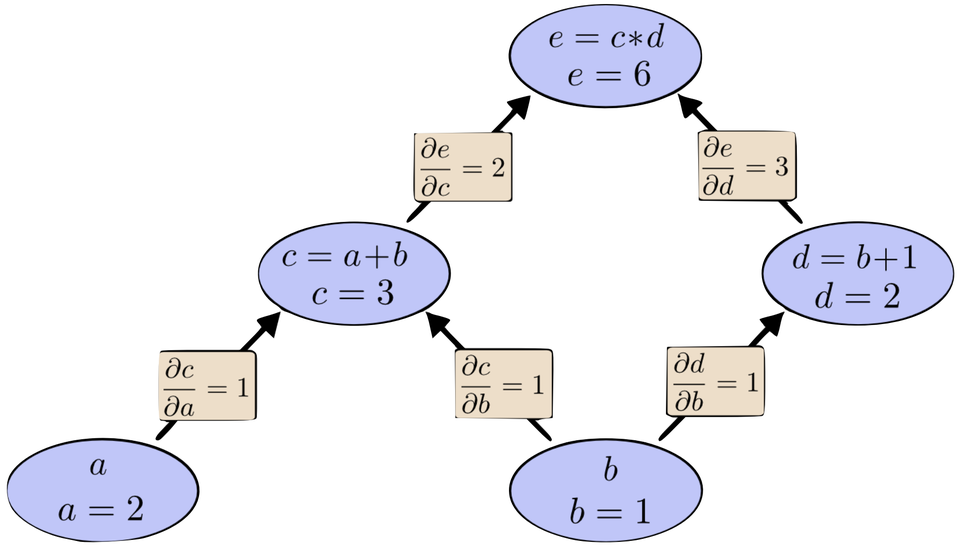
Derivatives on the graph

Example from this blog

Multiply derivatives in a path, and sum over paths

$$\frac{\partial e}{\partial b} = \frac{\partial e}{\partial c} * \frac{\partial c}{\partial b} + \frac{\partial e}{\partial d} * \frac{\partial d}{\partial b}$$

$$\frac{\partial e}{\partial a} = \frac{\partial e}{\partial c} * \frac{\partial c}{\partial a}$$



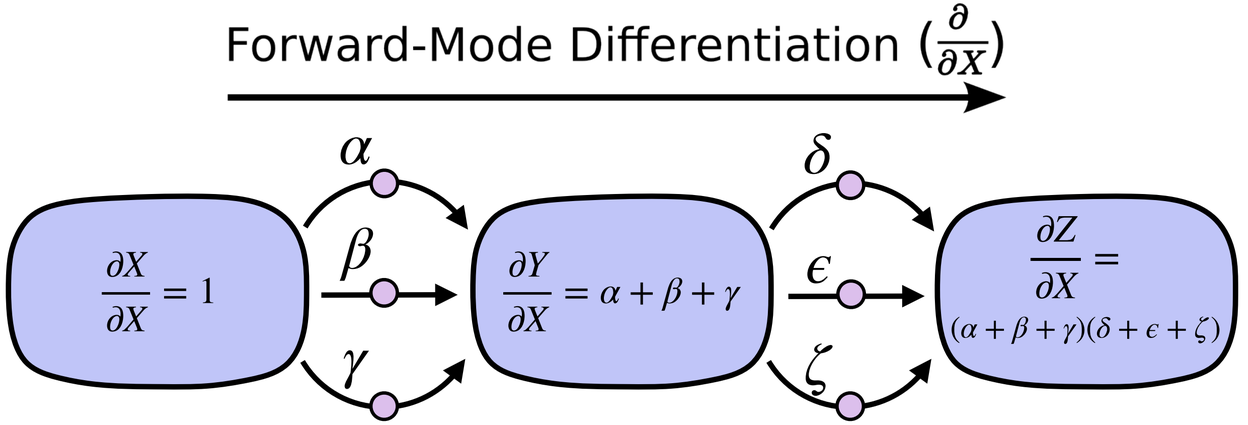
Number of terms can explode, but there is a more efficient way

Automatic differentiation

Forward and reverse-mode differentiation

Example from this blog

Forward-mode differentiation: derivative w.r.t. each input



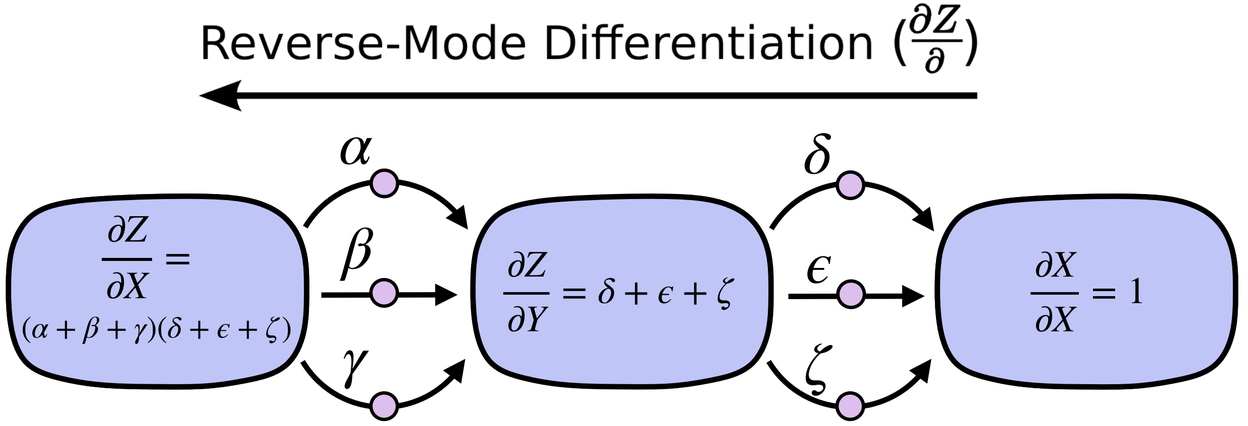
Efficient when we approach the extreme $\mathbf{f} : \mathbb{R} \to \mathbb{R}^m$ → $\left(\frac{\partial f_1}{\partial x}, \ldots, \frac{\partial f_m}{\partial x}\right)$

Automatic differentiation

Forward and reverse-mode differentiation

Example from this blog

Reverse-mode differentiation: derivative w.r.t. each output



Efficient when we approach the extreme $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}$ → $\left(\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f_m}{\partial x_n}\right)$

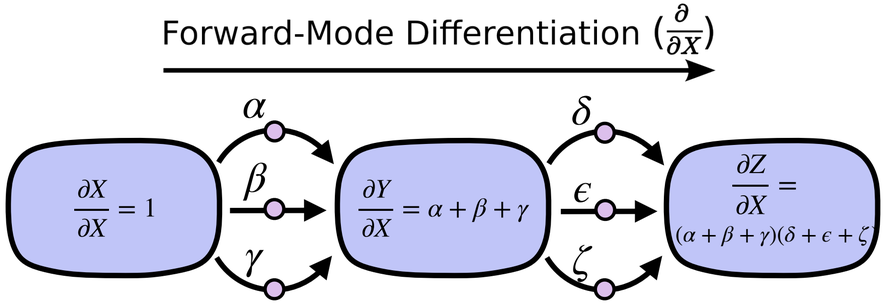
Automatic differentiation

Forward and reverse-mode differentiation

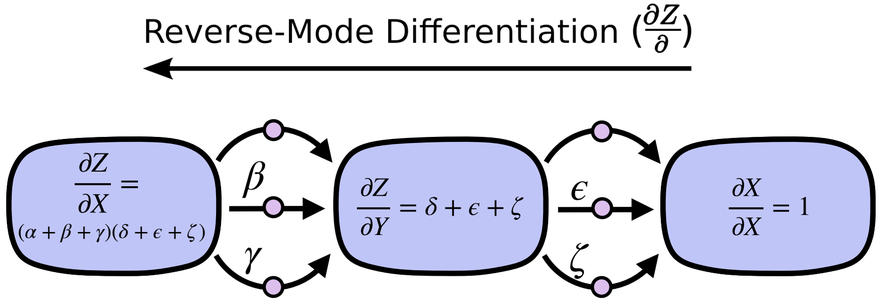
Example from this blog

In general, both modes evaluate the Jacobian

$$\mathbf{J}_f(\mathbf{x}) \in \mathbb{R}^{m \times n} \text{ for } \mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$$



Gradient from the Jacobian vector product (JVP) → directional derivatives



Gradient from the Vector-Jacobian product (VJP) → Total derivative